

```
POST /billpay/deletebillers.do HTTP/1.1
Host: www.fictitiousbank.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; de; rv:1.9)
Gecko/2008052906 Firefox/3.6.2t
...
...
Cookie: Hmdlkfchiefv52598542jrewr443
...
Content-Length: 12

userID=31396&requesttype=netbank
```

On forwarding this modified request to the server, it will respond with the *delete biller* page of `userID=31396` instead of `userID=31385`. Now, the attacker may be able to delete billers for another customer, resulting in a compromised operation over the victim's account.

Failure to restrict URL access

This happens when the developer fails to restrict access to administrative pages, privileged pages or other file resources because of the lack of a proper access control policy. An example: the *password change* functionality for *fictitiousbank.com*'s admin module is at <https://www.fictitiousbank.com/admin/passwordchange.do>. It's supposed to be accessed by admin users only, and only after a successful login. But, an external low-privileged user can easily guess and access it by entering the above URL in a browser. If successful, he can change the admin password and even prevent the admin logging in. This cannot happen if proper access restrictions were placed on the *passwordchange* module.

The above are the six most prevalent Web application vulnerabilities that HDIV covers. Instead of trying to mitigate them separately, all these can be mitigated by just installing and configuring HDIV, as I will now discuss.

Basic concepts and architecture

HDIV was mainly designed to transparently extend the security functionalities in Struts, without adding any complexity to the application development. To begin, it's important to understand one of the key concepts—*states*. For HDIV, a *state* represents all the data contained in the request. A *state* is made from parameters in the request, its values, and the page that is requested. For example, the *state* for the page <http://www.fictitiousbank.com/paybill/deletebillers.do> might be represented as follows:

```
Action: /paybill/deletebillers.do
Parameters:
  userID:
    values: 31385
    type: link
  requesttype:
    values: netbank
    type: link
```



Note: There can be more than one *state* for a page, depending on the links and forms contained in the page. While processing such a page, a *state* object is generated by HDIV for each request and link contained in the page.

The generated *states* can be stored at the client (mostly in hidden field values in encrypted or encoded format), server-side, or both. The whole working of HDIV is based on the later verification of requests sent by clients, by comparing the data sent by the client with the *state*. HDIV is designed with two main modules: Tag Library and Security Filter. Tag Library is responsible for modifying the HTML content sent to the client, and the Security Filter later validates the content sent by the client.

HDIV operational strategies

HDIV operates in any of the following three strategies: cipher strategy, hash strategy or memory strategy. Any of these can be used, depending on the need, and on server performance issues.

Cipher strategy

This strategy provides both data integrity and confidentiality to an insecure Web application. The steps followed by HDIV under cipher strategy are as follows.

First, HDIV collects all the request data (parameters, cookies, headers and URL fields) and generates an object of type *org.hdiv.state.IState*. If the confidentiality flag has been activated, then HDIV will replace non-editable values with relative values. For example: `userIDs 31385, 31386, 31387` will be replaced with `1, 23, 543`. Thus, HDIV guarantees the confidentiality of non-editable data. After the creation of *IState*, it is serialised, compressed, ciphered using a symmetrical algorithm, and then encoded in Base64, to guarantee integrity of data.



Note: Here, 'serialised' means the object is converted to a series of bytes, so it can be easily saved to persistent storage, or streamed across a communication link. The byte-stream can then be de-serialised into a replica of the original object.

This encoded *IState* is then sent to the client in a hidden form field, or as a parameter if it is a link. And it can be seen in a separate parameter, `_HDIV_STATE`. For example:

```
http://fictitiousbank/registration.do?userID=1&_HDIV_STATE=3238375318mxxzpkf
318mxxzpkfefwj374jgksg359

<input type="hidden" name="_HDIV_STATE_" value="3238375318mxxzpkf
efwj374jgksg359" />
```

Now, when a client makes a request, the encoded *IState* value is sent back to the server (HDIV), which then decodes and decrypts the value of `_HDIV_STATE`. If HDIV is not able to successfully decrypt it, then it assumes that modifications have been made in the request, and HDIV throws a customised error